RAJIV GANDHI COLLEGE OF ENGINEERING AND TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING

ASSIGNMENT - 3

Prepared by: Mohamed. B. Sirajuddeen

Reg No : 22TD0053

Subject Name : Embedded Systems

Subject Code : CST62

Year/Sem/Sec : IV/VI/B

Date of Submission: 10/4/2025

Unit : 3

Teacher's Sign :

Department Vision And Mission

Vision :

Cultivate student as technocrats researchers and entrepreneurs in the field of computer Science and engineering.

Mission :

+ The impact quality education in computer science and engineering by means of learning techniques and value-added courses.
+ To inculcate professionals excellence and research culture by encouraging projects in cutting techniques industry interactions.
+ To build leadership, skills ethical values and team works among the students.
+ To strengthen the collaboration of departments, and industry internships mentorships and professional body activities.

1. Explain the Thumb instruction set in ARM architecture How does it differ from the standard ARM instruction set in terms of instruction size, performance and application ?

Thumb Instruction Set

→ Thumb encodes a subset of the 32-bit ARM instructions into a 16 bit instruction set space.

→ Since, Thumb has higher performance than ARM on a processor with a 16 bit data bus, but lower performance than ARM on a 32-bit data bus. Use thumb for memory-constrained systems.

→ Thumb has higher code density - the space taken up in memory by an executable program - than ARM

→ For memory constrained embedded systems, for eg mobile phones and PDAs, code density is very important.

→ On average, a thumb implementation of the same code takes up around 30% less memory from the equivalent ARM implementation.

→ Even though the Thumb implementation uses more instructions, the overall memory footprint is reduced code density was the main driving focus for the thumb instruction set

→ Because it was also designed as a compiler target, rather than for hand-written assembly code, we recommend that you write Thumb-targeted code in a high-level language like C or C++

→ Each thumb instruction is related to a 32-bit ARM instruction. The simple thumb ADD instruction being decoded into an equivalent ARM ADD instruction

Eg: ARM code

```
ARM divide
; IN: r0 (value), r1 (divisor)
; OUT: r2 (Modulus), r3 (divide)

    MOV     r3, #0
loop
    SUBS    r0, r0, r1
    ADDGE   r3, r1, #1
    BGE     loop
    ADD     r2, r0, r1
```

5 × 4 = 20 bytes

Thumb code

```
Thumb divide
; IN: r0 (value), r1 (divisor)
; OUT: r2 (Modulus), r3 (Divide)

    MOV     r3, #0
loop
    ADD     r3, #1
    SUB     r0, r1
    BGE     loop
    SUB     r3, #1
    ADD     r2, r0, r1
```

6 × 2 = 12 bytes

→ The thumb instructions available in the THUMB 2 architecture used in the ARMV5TE architecture. Only the branch relative instruction can be conditionally executed.

→ The limited space available in 16 bit causes the barrel shifter operations, ASR, LSL, LSR and ROR to be separate instructions in the Thumb ISA.

Thumb Instruction Set

Mnemonics	THUMB ISA	Description.
ADC	V1	add two 32-bit values and carry.
ADD	V1	add two 32-bit values
AND	V1	Logical bitwise AND of two 32 bit values
ASR	V1	Arithmetic shift right
B	V1	Branch relative.
BIC	V1	Logical bit clear (AND NOT) of two 32-bit values
BXPI	V2	Breakpoint instruction.
BL	V1	relative branch with link.
BLX	V2	Branch with link and exchange.
BX	V1	branch with exchange.
CWN	V1	Compare negative two 32-bit value
CMP	V1	Compare two 32-bit integer.
EOR	V1	Logical exclusive OR of two 32-bit values

The comparison between the Thumb and ARM instruction sets in terms of instruction size, performance and application.

1. Instruction Size

Feature	ARM	Thumb
Instruction width	32 bits	16 bits (Thumb-1), mixed 16/32 bits (Thumb-2)
Code density	Lower (larger binaries)	Higher (smaller binaries)

→ ARM uses fixed 32-bit wide instructions.

→ Thumb uses compressed 16-bit instructions, which leads to smaller code size beneficial in systems with limited memory.

2. Performance

Feature	ARM	Thumb Instructions.
Execution Speed	Generally faster	Slightly slower (Thumb1) comparable (Thumb-2)
Instruction set Richess	Full feature set	Subset (Thumb-1), near-full (Thumb-2)

→ ARM mode allows more complex instruction and full register access, which may lead to higher performance.

→ Thumb-1 trades off some performance and functionality for compactness.

→ Thumb-2 improves this by introducing 32-bit instructions when needed, blending size and performance.

3. Application

Feature	ARM	Thumb
use case	Performance critical system	Memory-constained systems.
Typical targets	High-end processors (eg. cortex-A)	Embedded and low-power systems (eg: cortex-M)

→ ARM is typically used in high performance applications like smartphones and embedded linux systems.

→ Thumb is ideal for deeply embedded systems, microcontroller, and real time applications where memory and power efficiency are key.

2) Why is the Thumb instruction set preferred in embedded systems? Discuss with an example how thumb mode improves code density while maintaining performance?

The thumb instruction set preferred in embedded systems

1. Higher code density
   → Thumb instructions are 16-bit, compared to ARM's 32-bit instructions.
   → This means more code fits into the same memory space, which is crucial for embedded systems with limited Flash or ROM.
   → Smaller binaries also reduce cost and power consumption.

2. Power consumption

   Thumb instructions typically require less power to execute than ARM instructions, which is important in battery-powered embedded systems.

3. Lower power consumption
   → Fetching and decoding smaller 16-bit instructions use less power.
   → This is critical for battery-powered or energy sensitive applications (eg. wearables, IOT devices)

4. Adequate performance for many embedded Tasks

→ ~~For~~ Although thumb (especially thumb-1) is slightly slower than ARM, many embedded tasks don't need high performance.

→ Thumb-2 used in modern cortex-M cpu, provides a good balance between performance and size.

5. Cost efficiency

→ Smaller memory = cheaper microcontrollers

→ lower power consumption = cheaper batteries and longer device life.

6. Memory access

Thumb instructions often require fewer memory accesses than ARM instruction, which can improve performance and reduce power consumption.

7. Compiler optimization

Many compilers can optimize code for the thumb instruction set, which can result in smaller and more efficient code.

8. Efficient use of Instruction cache and Memory Bandwidth

→ More instructions fit in a cache line, and memory fetches are needed.

→ This leads to better instruction cache utilization faster execution in real time systems.

9. Supported by cortex-M cores

→ ARM cortex-M processor (widely used in embedded systems) only supported thumb or thumb-2, not the full 32-bit ARM instruction set.

→ These processors are optimized for Thumb execution making it the natural choice.

Thumb mode can improve code density while maintaining performance by using 16-bit instructions instead of 32-bit instructions.

Example

```
assembly
; ARM mode
loop:
    LDR R0, [R1] ; load value from memory
    ADD R0, R0, #1 ; Increment value
    STR R0, [R1] ; store value back to memory
    CMP R0, #10 ; Compare values to 10.
    BLT loop ; Branch if less than 10.
```

In ARM mode, each instruction is 32 bits long, resulting in a total code size of 20 bytes (5 instructions × 4 bytes per instruction)

Code size Comparison: ARM vs Thumb

Mode	Instruction set	Typical code size	Instruction count
ARM	32-bit	~12 bytes	3 instructions
Thumb	16-bit	~6 bytes	3 instructions

Even though both may execute similar instructions the thumb instructions ~~are~~ are half the size resulting in 50% reduction in cost size.

Performance maintenance

Despite the reduced code size, the performance of the loop remains the same. This is because the instruction are still executed in the same order and with the same functionality.

Benefits

→ Reduced memory usage.
→ Improved cache performance
→ lower power consumption.
→ Tighter, more efficient binaries...

3. Write an ARM thumb assembly code to perform the following operation.

R0 = (R1+R2) - (R3 x R4) R0 = (R1 + R2) - (R3 / times R4)

R0 = (R1+R2) - (R3 x R4). Explain the instruction sequenc and how thumb mode optimizer execution.

R0 = (R1+R2) - (R3 x R4)

```
ADD   R5, R1, R2    ; R5 = R1 + R2
MUL   R6, R3, R4    ; R6 = R3 x R4
SUB   R0, R5, R6    ; R0 = R5 - R6
```

explanation

→ Add computes R1 + R2 and stores the result in R5.
→ MUL multiplies R3 x R4 and store the result in R6.
→ SUB subtracts the result of multiplication from the addition and stores the final result in R0.

R0 = (R1+R2) - (R3) times R4)

```
push  {R5, R6}    ; same temporary ~~reg~~ registers
ADD   R5, R1, R2  ; R5 = R1+R2
MUL   R6, R3, R4  ; R6 = R3 x R4
```

```
SUB     R0, R5, R6    ; R0 = (R1+R2) - (R3*R4)
POP     {R5, R6}      ; Restore temporary registers.
```

constraints

→ R0 - R7 only in thumb → instructions.
→ MUL is available and operates on low registers (R0 - R7)
→ If R5 and R6 are used for other purpose you may need to save / restore them.

R0 = (R1+R2) - (R3 x R4)

```
ADD   R5, R1, R2    ; R5 = R1+R2
MUL   R6, R3, R4    ; R6 = R3 x R4
SUB   R0, R5, R6    ; R0 = R5 - R6
```

Explanation

→ ADD R5, R1, R2 → Adds R1 and R2, stores in temporary register R5.

→ MUL R6, R3, R4 → Multiplies R3 and R4, stores in temporary register R5.

→ ~~ADD~~ SUB R0, R5, R6 → subtracts the product from the sum and stores final result in R0.

Thumb Mode optimization

Thumb mode is a compact 16-bit instruction set for ARM processors, designed for better code density and performance in memory-constrained environments.

1. Smaller instruction size (16 bit vs 32-bit)

→ Saves memory space - more instructions fit in the same ROM / RAM
→ Reduces instruction fetch time from flash or memory.

2. Faster fetch-decode cycles

→ Smaller instructions mean the CPU can fetch more instructions per memory read.

→ Improves throughput, especially for low power or low frequency cores.

3. Efficient for low power devices

Less memory access = Less power consumed.

4. Low register usage (R0 - R7)

→ Thumb-1 limits most instructions to use low registers.

→ Keeps the instruction encoding short and simple.

5. Hardware Support

→ Many microcontrollers (like ARM CORTEX M0/M3) are optimized specifically for Thumb, not full ARM).

→ Thumb is often the default execution mode in microcontrollers.

4. What is software Interrupt (SWI) Instruction in Thumb code? How was it used to implement system calls?

Provide an example showing how SWI is used to invoke an OS service

The Thumb software interrupt (SWI) instruction causes a software interrupt execution.

If any interrupt or exception flag is raised in thumb state, the processor automatically retreats back to ARM state to handle the exception.

Syntax

SWI immediate

SWI	Software Interrupt	lr_svc = address of instruction following the SWI SPSR_svc = CPSR PC = vector + 0x3 CPSR mode = SVC CPSR I = 1 (mask IRQ interrupts) CPSR T = 0 (ARM state).

Implement system calls

1. User code Issues a system call:

→ The user program wants to access a privileged operation like reading a file or allocating memory.

SVC #<imm8>

→ The immediate value (like #0, #1 etc) tells the OS which system call is being requested,

2. Processor Trigger SWI exception

When the CPU executes the SVC instruction:

↳ It switches to supervisor mode (a privileged mode)

→ It saves the context (eg PC, CPSR, on the stack)

→ It jumps to the SVC handlers address (defined in the vector table)

3. SVC handler in the Kernel executes

The SVC handler reads the immediate value from the instruction to identify the system call.

→ It also reads any arguments passed (usually in register like r0, r1, etc)
→ Then it executes the corresponding system call function

4. Return the user mode
→ The handler restore the saved context.
→ Execution returns to usermode, continuing from the instruction after SVC.

Example

The processor goes from thumb state to ARM state after execution.

PRE

cpsr = nzcVqifT - USER
PC = 0x00030006
L = 0x003fffff ; lr = r14
r0 = 0x12
0x00080000 SWI 0x45

POST

cpsr = nzCVqfft - SVC
spsr = nzCVqifT - USER
pc = 0x00000008
lr = 0x00080002
r0 = 0x12

SWI assembly code

```
MOV   R0, #1    ; system call number) (eg: print)
MOV   R1, #42   ; Argument (eg: number to print)
SVC   #0        ; trigger the system call
```

SWI is used to Invoke an OS Service

```
AREA Reset, CODE, READONLY
    ENTRY

Start
    MOV r0, #'A' ; load ASCII 'A' into r0
    SWI 0x01     ; software interrupt to call OS
                   service # 0x01
    B
END
```

Explanation

→ MOV r0, #'A'

Load the ASCII value for 'A' into register r0.

→ SWI 0x01

Triggers a software interrupt. The processor:

✓ Switches to supervisor mode

✓ Saves the current context (eg: PC, CPSR)

✓ Jump to the SWI handle defined in the OS.

→ The SWI handler (in the OS kernel) check the immediate value (0x01) and knows to:

✓ Read r0

✓ Output the character 'A' to the console or UART

→ After handling, the OS restores context and returns control to user code.

b) Write short notes on single Register load and store Instruction and Multiple Register load and store Instruction in Thumb with appropriate example.

Single Register load-store Instructions

→ The thumb instruction set supports load and storing registers, or LDR and STR

→ These instruction use two preindexed addressing modes: offset by register and offset by immediate.

Syntax:

<LDR | STR> {<B | H>} Rd, [Rn, # immediate]

LDR {<H | SB | SH>} Rd, [Rn, Rm]

STR {<B | H>} Rd, [Rn, Rm>

LDR -RC, [PC, # immediate]

<LDR | STR> Rd, [SP, # immediate]

Addressing modes

Type	Syntax
load/store register base register + offset	[Rn, Rm]
Relative	[Rn, # immediate]
	[pc/sp, # immediate]

LDR	Load word into a register	Rd <- mem32[address]
STR	Save word from a register	Rd <- mem32[address]
LDRB	Load byte into a register	Rd <- mem8 [address]
STRB	Save byte from a register	Rd <- mem 8 [address]
LDRH	Load halfword into a register	Rd <- mem16 [address]
LDRH	Save halfword into a register	Rd -> mem16 [address]
STRH	Load signed byte into a register	Rd <- sign Extent (mem 8 [address])
LDRSB	Load signed byte halfword into a register	Rd <- signExtend (mem 16 [address])

Example

The two thumb instructions that use a pre index addressing mode. Both use the same pre-condition.

PRE mem 32 [0x90000] = 0x00000001 mem32[0x90004]

= 0x00000002 mem 32 [0x90003]

= 0x00000003

r0 = 0x00000000

r1 = 0x00090000

r4 = 0x00000004

LDR r0, [r1, r4] ; register

POST r0 = 0x00000002

r1 = 0x00090000

r4 = 0x00000004

LDR r0, [r1, #0x4]; immediate

POST r0 = 0x00000002

Both instructions carry out the same operation. The only difference is the second LDR uses a fixed offset, whereas the first one depend on the value in register r4.

Multiple - Register Load - store Instructions

→ The Thumb versions of the load-store multiple instructions are reduced forms of the ARM loadstore multiply instructions.

→ The only support the increment after (IA) addressing mode.

Syntax

<LDM | STM> IA Rn !, {low register list}

DMIA	Load multiple registers	{Rd}*N <- mem32[Rn+4*N], Rn = Rn+4*N
STMIA	Same multiple registers	{Rd}*N → mem32[Rn+4*N], Rn = Rn+4*N

→ Here N is the number of registers in the list of registers.

→ These instruction always update the base register.

→ Rn after execution.

→ The base register and list of register are limited to the low register r0 to r7.

Example The saves registers r1 to r3 to memory addresses 0x9000 to 0x900C. It also updates base register r4.

PRE

$r_1 = 0x00000001$

$r_2 = 0x00000002$

$r_3 = 0x00000003$

$r_4 = 0x9000$

STMA $r_4! = \{r_1, r_2, r_3\}$

POST

mem 32 [0x9000] = 0x00000001

mem 32 [0x9004] = 0x00000002

mem 32 [0x9008] = 0x00000003

$r_4 = 0x900c$